

tradeforme — Next Steps

Action list of things only **you** can do. Everything below is blocked on your input, your credentials, or your decision — not on more code.

Where the project stands

Repository	https://github.com/EtixP/tradeforme (private)
Branch	main
Milestones done	M1 (skeleton), M2 (DART ingestion), M4 (naive event study), M5 (strategy engine)
Tests passing	43 / 43
Disclosures in DB	111,622 across 60 trading days (Feb 27 – May 27, 2026)
Supply-contract events	533 “new contract” candidates (title-filtered)
Price data source	pykrx (free, no auth)
LLM extraction	Not yet wired — requires your API key
Broker integration	Not yet wired — requires your KIS credentials

Naive event study result (M4 first pass)

Computed event-time returns for all 533 title-filtered supply-contract disclosures. Returns measured from event-day close to T+1/T+2/T+5 close.

Horizon	n	Mean	Median	Std	Win%	P25	P75
1-day	533	+0.66%	−0.32%	6.60%	45.6%	−2.89%	+2.82%
2-day	533	+0.88%	−0.39%	10.60%	44.5%	−4.65%	+4.24%
5-day	533	+1.28%	−0.41%	14.11%	44.8%	−7.24%	+6.61%

Honest read: mean is positive but median is negative across all horizons, and the win rate is below 50%. A few large winners drag the mean up — the naive title filter does not appear to have a clean edge. This is the expected result: per CLAUDE.md, the real test is filtering to contracts where *contract_value* / *prior_year_revenue* ≥ 8%, which requires LLM extraction (Milestone 3).

Raw per-event CSV: `data/event_study_results.csv`

Action items

Sorted by priority. P1 unblocks the next milestone. P2 hardens the existing system. P3 is optional polish.

Priority	What	Why	How
P1	Rotate the DART API key	The current key briefly appeared in openai/pulsar during the 2023 SmoKE tests (2023-08-16). Replace it with a new one.	
P1	Get an LLM API key (Anthropic or OpenAI) for Milestone 3 (LLM extractions) with options to call platform openai.com or Anthropic.co	Required for Milestone 3 (LLM extractions) with options to call platform openai.com or Anthropic.co	
P1	Decide M3 vs more backfill first	M3 needs the LLM key above. Backfill is a more complex task. It is a more complex task.	
P2	Set up GitHub branch protection	Once live trading code exists, you don't want to accidentally break the system by pushing to the main branch.	
P2	Back up data/kdtb.db	111k rows of ingested DART data — easiest: rename it to kdtb.db.backup in the Desktop folder.	
P2	Schedule daily DART ingestion	Right now ingestion only runs when you click the CLI. It should be scheduled to run daily.	
P2	Obtain KIS broker credentials	Required for Milestone 6 (broker integration) — see KIS API for details.	
P2	Review risk thresholds in config/default.yaml	Current defaults match CLAUDE.md. Consider adjusting them to match the actual risk thresholds.	
P3	Install gitleaks pre-commit hook	Catches an accidental commit of .env or API keys. See gitleaks for details.	
P3	Decide if PDF action lists should be included in the PDF	Currently, the PDF will be generated only as a PDF. It should be generated as a PDF.	
P3	Decide on Korean market holiday handling	Right now, the ingestion CLI runs every day. It should be scheduled to run only on trading days.	

Where to find things

Project root	/Users/jsp2022310/Desktop/tradeforme
Repo on GitHub	https://github.com/EtixP/tradeforme
Project spec	CLAUDE.md (in repo root)
Source code	src/kdtb/ — schemas, config, data, strategy, risk, storage
Tests	tests/ — run with: .venv/bin/pytest
DART ingestion CLI	scripts/ingest_disclosures.py --date YYYY-MM-DD
Event study CLI	scripts/run_event_study.py [--limit N]
Local DB (gitignored)	data/kdtb.db — SQLite, inspect with sqlite3 or DB Browser
Event study CSV	data/event_study_results.csv
Secrets (gitignored)	.env — DART_API_KEY lives here
Config	config/default.yaml, config/paper.yaml, config/live_tiny.yaml
Live-trading guard	src/kdtb/config.py — raises if YAML says LIVE without ENABLE_LIVE_TRADING=true in

How to verify the project is healthy

Run these from the repo root any time you want to confirm nothing is broken:

```
.venv/bin/pytest
```

```
.venv/bin/python -m kdtb.main
```

```
.venv/bin/python scripts/ingest_disclosures.py --date $(date +%Y-%m-%d)
```

What Claude implemented in this loop

Modules added or completed:

- **src/kdtb/data/market_data_client.py** — pykrx wrapper for OHLCV fetching and event-time return calculation
- **scripts/run_event_study.py** — batched event study CLI with per-stock fetch optimization
- **src/kdtb/strategy/major_supply_contract.py** — Milestone 5 strategy engine (was placeholder)
- **tests/test_market_data_client.py** — 5 new test cases
- **tests/test_strategy.py** — 10 new test cases
- **scripts/generate_next_steps_pdf.py** — this PDF generator

Suite went from 28 to 43 passing tests. No regressions.

Honest caveats

- The naive event study uses title-only filtering. It does not separate small contracts from large ones, so noise dominates. M3 (LLM extraction) is needed to filter by contract/revenue ratio.
- The event date used is the DART receipt date (YYYYMMDD). Intraday timing of the disclosure is not

captured. If a disclosure is filed at 14:00 KST, our 'T+1' return actually includes the price reaction from 14:00 to next-day close, not a clean next-day-only return.

- Returns are raw close-to-close, not benchmark-adjusted. A market-wide rally on event days would inflate the mean.
- pykrx scrapes the KRX website. It's free but can be rate-limited. We sleep 0.15s between calls.
- LIVE_TINY broker code is not yet implemented. Even with KIS credentials, no real orders can be placed yet.